情報処理安全確保支援士試験　科目A-2　中分類23 情報セキュリティの脅威　#03

Webアプリケーションへの攻撃

入力が「命令」に化けるとき、何が起きるのか。根本的な対策と保険的な対策を分けて覚える

各ページの下に、根拠にした資料（IPA の支援士試験シラバス案・安全なウェブサイトの作り方、OWASP など）を書いた。過去問は IPA 公開の午前 II 。例のサイトや URL は Claude が作った

全体像 シラバス案 中分類23 小分類2「攻撃手法」アプリケーションの不備を狙った攻撃

1 サーバ側で命令に化ける
SQL インジェクション・OS コマンドインジェクション・HTTP ヘッダインジェクション・ディレクトリトラバーサル・バッファオーバーフロー・SSRF

2 ブラウザを巻き込む
XSS（反射型・格納型・DOM 型）・CSRF・クリックジャッキング・オープンリダイレクト・ドライブバイダウンロード

3 根本的な対策と保険的な対策
静的プレースホルダ・エスケープ・CSRF トークン・X-Frame-Options

過去の午前 II での出方（令和元〜7年度）

テーマ	出た回
CSRF の対策で効果がないもの	H31春・R6春・R7秋
ログから OS コマンドインジェクション	R2・R4春・R5秋
クリックジャッキング → X-Frame-Options	R4秋・R6秋
SQLi の実装／実装以外の対策	R1秋・R5春
攻撃と対策の組合せ	R3春

対策の取り違えを突く選択肢が定番。「その対策はどの攻撃のものか」を言えるようにする。

出典：IPA「支援士試験 シラバス（案）Ver.0.1」中分類23 小分類2／IPA 午前 II 各回（このPDFの問題演習を参照）

インジェクションの共通点：データの中に「区切り」を入れる

アプリが文字列をつないで命令を作ると…

```
命令の型   SELECT * FROM users WHERE id='入力'
入力       ' OR '1'='1' --
できる命令 SELECT * FROM users WHERE id='' OR '1'='1' --'
```

> 入力の中の ' が「データの終わり」と読まれ、そのあとが**命令**として動く。

攻撃	区切りになる文字	どこで命令になるか
SQL インジェクション	' ; --	データベース
OS コマンドインジェクション	; | && `	シェル
HTTP ヘッダインジェクション	改行 %0d%0a	HTTP 応答
メールヘッダインジェクション	改行	メールのヘッダ
XSS	< > "	閲覧者のブラウザ

出典：IPA「安全なウェブサイトの作り方 改訂第7版」1.1～1.7／OWASP「OWASP Top 10:2025」A05 Injection

第1章

サーバ側で命令に化ける

ログに残った文字列から、攻撃の種類を当てられるようにする。

SQL インジェクション：認証の回避から情報の窃取まで

よくある入力と結果

入力	起きること
`' OR '1'='1' --`	条件が常に真。パスワードなしでログイン
`' UNION SELECT id,pass FROM users --`	別の表の中身を画面に出す
`'; DROP TABLE orders; --`	表を削除（複文が通る環境）
`' AND SUBSTRING(pass,1,1)='a`	ブラインド：応答の差で1文字ずつ推測

ブラインド SQL インジェクション：結果が画面に出なくても、ページの違いや応答時間の差で情報を抜ける。**エラーを隠すだけでは防げない。**

R5春 問17：実装の対策＝プレースホルダ、実装以外の対策＝**DB アカウントの権限を最小限に。**ほかの選択肢は OS コマンド（シェルを起動しない・chroot）、セッション（乱数の ID・TLS）、トラバーサル（ファイル名を受け取らない）の対策。

出典：IPA「安全なウェブサイトの作り方」1.1 SQL インジェクション／IPA 午前 II 令和5年度春期 問17（令和元年度秋期 問17 と同じ）

静的プレースホルダ：値が命令にならないしくみ

文字列連結（危ない）
"... WHERE id='" + 入力 + "'" → 入力しだいで構造が変わる

↓

動的プレースホルダ
ライブラリが値をエスケープして SQL 文を組み立てる。ライブラリの不備が残りうる

↓

静的プレースホルダ（推奨）
① SQL の型を DB に送って構造を確定 → ② 値だけを送る。値は構造に影響しない

```
-- ① 先に構造を確定させる
PREPARE q FROM
 'SELECT * FROM users WHERE id = ?';
-- ② 値だけを渡す
EXECUTE q USING @input;
```

根本的な対策＝脆弱性そのものをなくす。**保険的な対策**＝攻撃されたときの被害を減らす（エラーを出さない、権限の最小化、WAF）。

R3春 問12 の正解は「SQL インジェクション＝**静的**プレースホルダ」。

出典：IPA「安全なウェブサイトの作り方」1.1・別冊「安全な SQL の呼び出し方」／IPA 午前 II 令和3年度春期 問12

OS コマンドインジェクション：ログから見抜く

R4春 問1（R2 問1 とほぼ同じ）のログ

```
GET /cgi-bin/submit.cgi?user=;cat%20/etc/passwd HTTP/1.1
Accept: */*
Accept-Language: ja
Host: test.example.com
```

1. `%20` は空白 → `;cat /etc/passwd`
2. `;` でコマンドを区切り、`cat`（ファイルを表示）を実行
3. SQL の断片も `<script>` も改行もない → **OS コマンドインジェクション**

R5秋 問1：入力が PHP の **exec 関数**に渡されて、シェルスクリプトが動く → OS コマンドインジェクション。

対策	種類
シェルを起動する関数（exec・system・Perl の open など）を使わない	**根本**
どうしても使うなら、引数に入る値を許可リストで検査	**根本**
実行権限の最小化、chroot、WAF	**保険**

出典：IPA 午前 II 令和4年度春期 問1・令和5年度秋期 問1／IPA「安全なウェブサイトの作り方」1.2 OS コマンド・インジェクション

ヘッダインジェクションとディレクトリトラバーサル

HTTP ヘッダインジェクション（HTTP レスポンス分割）

```
入力 lang=ja%0d%0aSet-Cookie:%20sid=attacker
↓ 応答
Location: /top?lang=ja
Set-Cookie: sid=attacker   ← 偽のヘッダが増える
```

改行で**ヘッダや本文を追加**。偽の cookie（セッション固定化）、偽ページ、キャッシュ汚染。対策は**ヘッダ出力用の API**を使い改行を許さない。メールフォームで同じことをすると**メールヘッダインジェクション**（Bcc を足して迷惑メールの踏み台に）。

ディレクトリトラバーサル（パストラバーサル）

```
download.php?file=..%2F..%2F..%2Fetc%2Fpasswd
/var/www/files/../../../etc/passwd
→ /etc/passwd を読み出される
```

対策	種類
ファイル名を直接受け取らない（番号で指定）	**根本**
ファイル名からディレクトリ部分を取り除く	**根本**
重要なファイルを公開領域に置かない	**保険**

出典：IPA「安全なウェブサイトの作り方」1.3 パス名パラメータの未チェック／ディレクトリ・トラバーサル・1.7 HTTP ヘッダ・インジェクション・1.8 メールヘッダ・インジェクション

バッファオーバーフロー：戻り先を書き換える

スタックのイメージ（上ほど後から積む）

領域	正常	攻撃後
ローカル変数 buf[16]	名前	AAAA…（16字超）
カナリア（あれば）	0x5e9a…	壊れる → 検出
戻り先アドレス	呼び出し元	攻撃コードの位置

> C 言語の `strcpy`・`gets` のように長さを確かめない関数が原因になりやすい。

OS・コンパイラ側の緩和策	何を邪魔するか
DEP（NX ビット）	データ領域に置いたコードの実行
ASLR	攻撃コード・ライブラリの位置の予測
スタックカナリア	戻り先を書き換えたまま戻ること
安全な関数・境界の確認	そもそもあふれさせない（根本）

出典：IPA「支援士試験 シラバス（案）」中分類23 用語例「バッファオーバーフロー」・中分類26「バッファオーバーフロー対策」。スタックの図は Claude が単純化して作成

SSRF：サーバに内部へ取りに行かせる



ファイアウォールの内側にいる**サーバ自身**がリクエストを送るので、外からは届かない資源に届く。クラウドではメタデータサービスから**一時的な認証情報**が盗まれる。

OWASP Top 10:2025 では **A01 アクセス制御の不備**に統合された。

対策
取りに行く先を許可リストで限る
プライベートアドレス・リンクローカル（169.254.0.0/16）への接続を拒否
リダイレクトをたどらない、名前解決後の IP でも確認
メタデータにトークン必須の方式（AWS の IMDSv2 など、#18）

出典：OWASP「Server Side Request Forgery Prevention Cheat Sheet」「OWASP Top 10:2025」／IPA「支援士試験 シラバス（案）」中分類26「インスタンスメタデータサービスの保護」

第2章

ブラウザを巻き込む攻撃

被害者のブラウザが、攻撃の道具にされる。

XSS の3つの型：スクリプトはどこから来るか

型	スクリプトの通り道	サーバに保存	被害者がすること
反射型	罠のリンク → サーバの応答にそのまま返る → 実行	**されない**	**罠のリンクを開く**
格納型	掲示板などに投稿 → DB に保存 → 見た人全員に配られる	**される**	**ページを見るだけ**
DOM 型	ブラウザの JavaScript が URL の一部を innerHTML に書き込む	**されない（#以降は送られない）**	**罠のリンクを開く**

```
// DOM 型の例（危ない）
el.innerHTML = location.hash.slice(1);
// 安全：HTML として解釈しない
el.textContent = location.hash.slice(1);
```

できること：**cookie（セッション ID）の窃取**、偽の入力フォーム、画面の書き換え。**そのサイトの権限**で動くのが怖さ。

出典：IPA「支援士試験 シラバス（案）」用語例「クロスサイトスクリプティング（反射型，格納型，DOM 型）」／OWASP「DOM based XSS Prevention Cheat Sheet」

XSS の対策：出力の直前に、出力先に合わせてエスケープ

文字	置き換え後
<	<
>	>
"	"
'	'
&	&

対策	種類
出力するすべての値をエスケープ（属性値は " で囲む）	根本
URL は http/https で始まるものだけ出力	根本
外部のスタイルシートを取り込まない	根本
cookie に HttpOnly（JavaScript から読めない）	保険
CSP でスクリプトの読み込み元を制限（#17）	保険
入力値の検証	保険

R3春 問12：「XSS の対策＝任意の外部サイトのスタイルシートを取り込めるようにする」は逆。危険が増える。

出典：IPA「安全なウェブサイトの作り方」1.5 クロスサイト・スクリプティング／IPA 午前 II 令和3年度春期 問12

CSRF：ログイン中のブラウザに送らせる

① 利用者が正規サイトにログイン
cookie にセッション ID

→

② 罠のページを開く
自動で送信するフォームが仕込まれている

→

③ 正規サイトへ送金リクエスト
ブラウザが cookie を自動で付ける

→

④ 本人の操作として処理
送金・書き込み・設定変更

R7秋 問16 の選択肢	効果
重要な操作の都度パスワードを入力させる	ある
レスポンスボディの秘密の値と送付値を照合	ある
Referer で正しいリンク元か確認	ある
< → < などの置き換え	ない（XSS の対策）

CSRF のリクエストには**特殊文字がいらない**。だからエスケープは効かない。H31春・R6春・R7秋の3回とも正解はこれ。

SameSite 属性の cookie は、ほかのサイトからのリクエストに cookie を付けないので有効（#17）。

出典：IPA 午前 II 令和7年度秋期 問16（令和6年度春期 問1・平成31年度春期 問10 とほぼ同じ）／IPA「安全なウェブサイトの作り方」1.6 CSRF

CSRF トークンが効く理由

正規のフォーム
<input type="hidden" name="token" value="9f3c…（乱数）">

↓ 送信

サーバ
セッションに覚えた値と一致？ → 処理する

罠のページ
正規ページの中身を読めない（同一オリジンポリシー）→ token を付けられない
→ 拒否

条件：推測できない乱数で作る、セッションごと（または操作ごと）に発行、GET の URL に載せない（Referer やログから漏れる）。

R3春 問12：「CSRF の対策＝リクエストに GET を使う」は逆。GET は画像タグやリンクで簡単に送らせられる。重要な操作は POST＋トークン。

出典：IPA「安全なウェブサイトの作り方」1.6 CSRF 根本的解決／IPA「支援士試験 シラバス（案）」中分類26 用語例「CSRF トークン」

クリックジャッキングとオープンリダイレクト

クリックジャッキング

上：正規サイトの「退会する」ボタン（透明な iframe）

下：罠の「プレゼントを受け取る」ボタン

R6秋 問14 の選択肢	効く攻撃
cookie に HttpOnly	XSS の cookie 窃取
cookie に Secure	盗聴
Strict-Transport-Security	HTTP への格下げ
X-Frame-Options	クリックジャッキング

オープンリダイレクトの悪用

```
https://bank.example.jp/login?next=https://evil.example.net/
```

ドメインは本物なので信用されやすく、**フィッシング**に使われる。対策は転送先を固定するか許可リストで限る。

ドライブバイダウンロード

サイトを**見ただけ**でマルウェアが入る。改ざんされた正規サイト・悪意のある広告 → エクスプロイトキット（#01）→ ブラウザやプラグインの脆弱性。

出典：IPA 午前 II 令和6年度秋期 問14（令和4年度秋期 問11 と同じ）／IPA「安全なウェブサイトの作り方」1.9 クリックジャッキング／IPA「支援士試験 シラバス（案）」用語例

第3章

攻撃と対策の組合せ

選択肢のひっかけは「別の攻撃の対策」。表で整理する。

攻撃 × 対策の対応表

攻撃	根本的な対策	保険的な対策	取り違えやすい対策
SQL インジェクション	静的プレースホルダ	エラーを出さない・DB 権限の最小化・WAF	エスケープだけ
OS コマンド	シェルを起動しない	実行権限の最小化・chroot	プレースホルダ
トラバーサル	ファイル名を直接受け取らない	重要ファイルを公開領域に置かない	chroot
HTTP ヘッダ	ヘッダ出力用 API・改行を許さない	—	HSTS
XSS	出力時のエスケープ	HttpOnly・CSP・入力の検証	CSRF トークン
CSRF	秘密のトークン・再認証	Referer・SameSite	特殊文字の置換
クリックジャッキング	X-Frame-Options・frame-ancestors	重要操作の再確認	HttpOnly・Secure・HSTS
セッションハイジャック	乱数の ID・ログイン時に再発行・TLS	Secure・HttpOnly	固定の ID（R3春）

出典：IPA「安全なウェブサイトの作り方 改訂第7版」第1章 各節の「根本的解決」「保険的対策」を Claude が表にまとめた

過去問の問われ方のまとめ

ログを読む: `;cat` → OS コマンド、`../` → トラバーサル、`%0d%0a` → ヘッダ、`' OR` → SQLi、`<script>` → XSS。

効果がないものを選ぶ: CSRF に特殊文字の置換。答えは毎回同じ。

組合せ: 4つとも「攻撃」は正しく、「対策」の正誤を見る。逆効果の対策（外部 CSS を許す・GET を使う・固定 ID）を混ぜてくる。

科目B（午後）での出方: 脆弱性診断の結果表、ソースコードの一部、ログから「どの脆弱性か」「どう直すか」を答える。プレースホルダへの書き換えやエスケープ箇所の指摘が定番（#24 事例演習）。

ブラウザ側の防御（SameSite・CSP・CORS・HSTS・同一オリジンポリシー）は **#17 セキュア開発と Web の防御**で詳しく。

出典：IPA 午前 II 令和元〜7年度（このPDFの問題演習）／IPA「情報処理安全確保支援士試験 科目B サンプル問題」